

RANGE CONSOLE

Using Range

Type these at the `range>` prompt on ocelot, after `range` has planted the agents. Select one host with `use` before a play, a file copy, or a shell. The id is the last octet of the address. Host `172.24.10.180` is `180`.

`exit` at `range>` stops the operator. `exit` inside `shell` only returns to `range>`.

Start, watch, and check

range

```
range
```

Opens the WAN master if it is down, copies your key, starts one agent on each host, then becomes the operator. Leave this window open. The prompt is `range>`.

```
● ● ● ocelot

$ range
Opening the path to the WAN router. Type that password.
Installing keys and starting an agent on each host. The id is the last octet.
Starting 5 agent(s). The id is the last octet of the address.
Starting the operator. This window is now the range console.
ange>
```

range plant --skip

```
range plant --skip 172.24.18.82
```

Same plant, but that address is left alone. Repeat the flag, or separate addresses with commas. The line `Extra plant arguments` means this copy of `range` accepted the flag. If that line is missing, the script on ocelot is old.

```
● ● ● ocelot

$ range plant --skip 172.24.18.82
Extra plant arguments: --skip 172.24.18.82
skipping 172.24.18.82
starting 4 agent(s). The id is the last octet of the address.
```

range watch

```
range watch
```

A second terminal. Shows who is logged in on the focused hosts. This is not the agent list. Routers can appear here. Agents appear only in `sessions`.

```
● ● ● ocelot

$ range watch
180  abdullayev  pts/0  10.50.11.4
44   student     tty1   local
```

selftest

```
python3 range.py selftest
```

Checks TCP, HTTP, DNS, MQTT, and WebSocket on this machine, prints `PASS`, and exits. It does not start the operator. ICMP is skipped unless the shell can open a raw socket.

```
● ● ● ocelot

$ python3 range.py selftest
PASS tcp,http,dns,mqtt,ws
```

Pick a host

sessions

```
sessions
```

Lists agents that have called in. `up` means the channel is alive. TCP, WebSocket, and MQTT stay listed while the socket is up. A blank list means no agent has checked in yet.

```
● ● ● range

range> sessions
id      channel  state  profile  os      user      age
180     ws       up     continuous linux   abdullay 4s
44      mqtt     up     continuous linux   student  9s
```

ports

ports

Prints the ports this operator is listening on, and the DNS zone. Class ports are 443, 80, and 53. MQTT is 1883. WebSocket is 8070.

```
● ● ● range  
  
range> ports  
tcp 443  
http 80  
dns 53  
mqtt 1883  
ws 8070  
icmp echo (raw)  
dns zone lab
```

use

use 180

Selects one agent. The prompt becomes `range 180>`. Later commands go to that host only. A plain command, such as `hostname`, runs in `bash` on Linux and `PowerShell` on Windows. `cd` sticks for the next command.

```
● ● ● range  
  
range> use 180  
using 180  
range 180> hostname  
web-nginx  
[rc=0]
```

back

back

Clears the selection. The prompt returns to `range>`.

```
● ● ● range  
  
range 180> back  
ange>
```

profile

```
profile continuous
```

```
profile off
```

`continuous` is the class. TCP, WebSocket, and MQTT stay connected either way, so a command lands as soon as you type it. HTTP, DNS, and ICMP ask about once a second. `off` stops that beacon. Those three look again in about half an hour, which is how you turn them back on without a login. `live` means continuous. Do not use `hunt`.

```
● ● ● range

range 180> profile off
180 profile off queued
range 180> profile continuous
180 profile continuous queued
```

broadcast

```
broadcast hostname
```

Runs one command on every agent that is up. Routers do not answer. A host you skipped at plant does not answer.

```
● ● ● range

range> broadcast hostname
---- 180 ----
web-nginx
[rc=0]
---- 44 ----
deb9-44
[rc=0]
```

A normal command

```
id
```

Anything that is not a built-in word is sent to the selected agent. You must use a host first. [rc=0] means the command exited cleanly.

```
● ● ● range  
  
range 180> id  
uid=0(root) gid=0(root) groups=0(root)  
[rc=0]
```

A shell on the guest

shell

```
shell
```

One command at a time. The working directory sticks. nano will not work, because there is no terminal. exit returns to range> and does not stop the operator.

```
● ● ● range  
  
range 180> shell  
line shell on 180. Each line is one command. cd sticks. nano needs tty. exit returns.  
180$ pwd  
/root  
180$ exit  
ange 180>
```

tty

```
tty
```

A real bash on the guest. `nano` and `vi` work. Type `.quit` and press Enter to leave. `~~` and Enter also leaves. Ctrl-C goes to the guest, not to Range. Use TCP, WebSocket, or MQTT. On DNS, HTTP, or ICMP the same terminal exists, but it redraws slowly.

```
● ● ● range

range 180> tty
remote terminal. Type .quit and press Enter to leave.
root@web-nginx:~# nano /var/www/html/index.html
root@web-nginx:~# .quit
ange 180>
```

Files

get

```
get /tmp/range-test
```

```
get /var/www/html/index.html ./page.html
```

Copies a file from the guest to ocelot. If you omit the local name, it uses the file's basename in the current directory. Refuses a file over 8MB.

```
● ● ● range

range 180> get /tmp/range-test
got 3 bytes -> range-test
```

put

```
put range-test /tmp/range-test-back
```

Copies a local file onto the guest. The remote path must start with /. It clears the immutable bit first, so you can replace a locked page. Prefer TCP, WebSocket, or MQTT for anything large. The limit is 1.5MB.

```
● ● ● range  
range 180> put range-test /tmp/range-test-back  
WROTE=/tmp/range-test-back BYTES=3
```

The web page

Run `play snapshot` before a practice deface. `play revert` puts that copy back. `play undo` puts back the copy taken by the last deface, which is a different file.

play look

```
play look
```

Shows the web root, the file mode, the owner, the lock bits from `lsattr`, and the top of `index.html`. It does not change the host.

```
● ● ● range  
range 180> play look  
WEBROOT=/var/www/html  
MODE=644 OWNER=root GROUP=root MTIME=...  
---- head ----  
DONE  
[rc=0]
```

play snapshot

```
play snapshot
```

Saves the live page and any images next to it under `/var/lib/.hold/site/`. This is the copy `play revert` uses.

```
● ● ● range

range 180> play snapshot
SAVED=/var/lib/.hold/site/index.html
DONE
[rc=0]
```

play deface

```
play deface
```

```
play deface nobackup
```

Replaces `index.html` with the page from the kit and locks it. On Linux the lock is `chattr +i`. Editing then returns "Operation not permitted". The image is written beside the page and locked with it. A copy of the previous page is kept outside the site. `nobackup` skips that copy.

```
● ● ● range

range 180> play deface
WEBROOT=/var/www/html
IMAGE=/var/www/html/3ntity-logo.png
DONE
[rc=0]
```

play undo

```
play undo
```

Clears the lock and puts back the copy taken by the last deface. If there is no copy, it writes a plain index. This is not the snapshot.

```
● ● ● range  
  
range 180> play undo  
RESTORED=/var/www/html/index.html  
DONE  
[rc=0]
```

play revert

```
play revert
```

Clears the lock and puts back the snapshot from before the practice deface. Use this after a drill. `NO_SNAPSHOT` means you did not run `play snapshot` first.

```
● ● ● range  
  
range 180> play revert  
REVERTED=/var/www/html/index.html  
DONE  
[rc=0]
```

History, a message, and timestamps

play history

```
play history
```

```
play history abdullayev.coshgun
```

Shows that user's bash history and the lines that look like hunting. With no user, it shows root. A user who has never logged in has no history file. That is not an error.

```
● ● ● range  
  
range 180> play history  
LOOKING_FOR=root  
DONE  
[rc=0]
```

play history clear

```
play history backup clear
```

Erases that user's history files and root's history files. The home comes from passwd. backup lives in /var/backups, not /home/backup. ABSENT means the file was never there.

```
● ● ● range  
  
range 180> play history backup clear  
ABSENT=/var/backups/.bash_history  
CLEARED=/root/.bash_history  
DONE  
[rc=0]
```

play wall

```
play wall The page changed. The process list is the wrong tool.
```

Sends the rest of the line to every terminal on that host. `WALL=nobanner` means it used `wall -n`, so the students do not see a banner with your account. The words after `wall` are the message. They are not a username.

```
● ● ● range  
  
range 180> play wall The page changed. The process list is the wrong tool.  
WALL=nobanner  
DONE  
[rc=0]
```

play stamp

```
play stamp
```

```
play stamp /usr/bin/ps /bin/ls
```

Runs `touch -r` so the files the plays changed match the time of a system file. The default reference is `/bin/ls`. With no path, it stamps the files Range already touched.

```
● ● ● range  
  
range 180> play stamp  
STAMPED=/usr/bin/ps  
DONE  
[rc=0]
```

An account that can log in

play account

```
play account backup
```

The name must be mail, games, news, or backup. On Debian these already exist. The play sets a login shell, adds the account to sudo, and sets a shadow hash. The password is printed only on this console. It is not in the tasking the students can read. backup's home is /var/backups.

```
● ● ● range

range 180> play account backup
login for backup is local only (not in the tasking): (printed here)
EXISTS=backup
GROUP=sudo
SHADOW=hash
DONE
[rc=0]
```

play cloak

```
play cloak
```

Copies bash over false and nologin, and keeps the originals in /var/lib/.hold. A user whose shell is nologin can still get a shell. SAME_FILE means that path was already the same file, which is normal when /bin is a link to /usr/bin.

```
● ● ● range

range 180> play cloak
BACKUP=/var/lib/.hold/bin/false.orig
CLOAKED=/usr/bin/false
CLOAKED=/usr/sbin/nologin
DONE
[rc=0]
```

play uncloak

```
play uncloak
```

Puts `false` and `nologin` back from the hold directory. After this, `file` on those paths should not say they are `bash`. Run `play cloak` again only if that host should keep the puzzle for class.

```
● ● ● range

range 180> play uncloak
RESTORED=/usr/bin/false
RESTORED=/usr/sbin/nologin
DONE
[rc=0]
```

play veil

```
play veil abdullayev.coshgun
```

Wraps `ps`, `ss`, `netstat`, `w`, and `who`. Their output hides that account, the short `ps` form (`abdulla+`), the short `netstat` form (`abdulla`), the agent name, the hold path, and the attacker address. Ordinary services stay. `cupsd` is not hidden. `htop`, `top`, and `last` are not wrapped. Running it again is safe. The originals stay in `/var/lib/.hold/bin/`.

```
● ● ● range

range 180> play veil abdullayev.coshgun
COPIED=/usr/bin/ps
VEILED=/usr/bin/ps
VEILED=/usr/bin/netstat
DONE
[rc=0]
```

play unveil

```
play unveil
```

Puts those five binaries back. `w` on Debian 9 is rebuilt from `who` and `ps` if the original binary was already gone.

```
● ● ● range

range 180> play unveil
RESTORED=/usr/bin/ps
RESTORED=/bin/netstat
DONE
[rc=0]
```

play shell

```
play shell backup bash
```

Changes only the shell field in `/etc/passwd` to `vi`, `python`, `perl`, or `bash`. It does not replace `nologin`. That is `play cloak`. You do not need this play for the class.

```
● ● ● range

range 180> play shell backup bash
BEFORE=backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
AFTER=backup:x:34:34:backup:/var/backups:/bin/bash
SHELL=/bin/bash
DONE
[rc=0]
```

Leave these until after class

These change the host the students will see. Do not run them during the exercise unless that is the point of the hour.

play persist

```
play persist
```

Installs `mailq-local.service` so the agent starts again after a reboot. `PERSIST=cron` means that host has no `systemd` and a `cron` entry was used instead.

```
● ● ● range

range 180> play persist
PERSIST=systemd
UNIT=/etc/systemd/system/mailq-local.service
DONE
[rc=0]
```

play unpersist

```
play unpersist
```

Removes that service. Safe to run if you never persisted. The line is UNPERSIST=done.

```
● ● ● range  
  
range 180> play unpersist  
UNPERSIST=done  
DONE  
[rc=0]
```

play boot

```
play boot abdullayev.coshgun
```

Logs that user off this host. It refuses root. The student loses the session they are working in.

```
● ● ● range  
  
range 180> play boot abdullayev.coshgun  
DONE  
[rc=0]
```

play logs

```
play logs
```

```
play logs all
```

Empties auth, syslog, nginx, apache, wtmp, btmp, lastlog, and the journal. The files stay, and they are empty, which is the point. Bash history is left so you can still read what the students hunted. `play logs all` wipes that history too. On Windows this clears Application, System, Security, and PowerShell.

```
● ● ● range  
  
range 180> play logs  
DONE  
[rc=0]
```

```
● ● ● range  
  
range 180> play logs all  
CLEARED=/root/.bash_history  
DONE  
[rc=0]
```

exit

```
exit
```

Stops the operator. The listeners close, and ordinary pings to ocelot work again. Agents keep running on the guests until the VM is rebuilt or you stop them. `quit` is the same command.

```
● ● ● range  
  
range 180> exit
```